Do we need more bikes?
Project in Statistical Machine Learning

Ebba Brunvall

Hugo Eklöf

Eskil W. Forss

Åke Lindström

Abstract

In this project, we wanted to know if there was a need for more bicycles in Washington DC. We were given a dataset containing different temporal and meteorological data as well as a data query for low or high demand for bicycles. We used classification methods such as logistic regression, k-nearest neighbour, random forest, and boosting. We analysed the features and concluded that some features were unnecessary and some were dependent on one another. We also used cross-validation to determine the performance of each model. After comparing each model to one another, we concluded which model we used in our final analysis based on their accuracy and F2-score. Our final choice was the boosting model since it had the highest F2-score and the second highest accuracy, but only by a small margin after random forest.

1 Introduction

The company Capital Bikeshare owns a public bicycle-sharing system that serves Washington, D.C. The demand for bicycles is sometimes higher than the number of bicycles provided by the company. The goal of this project is to conclude if there is a need for more bikes in the city and to predict when there is a high demand. To do this, various classification models were trained on the huge training set from the city. The dataset includes various temporal and meteorological data, for example, the hour of day and the current temperature outside. In this report, four models were used: logistic regression, k-nearest neighbours, random forest, and boosting. These models were then compared after they had been trained on the data, and we decided on the most optimal one for our problem.

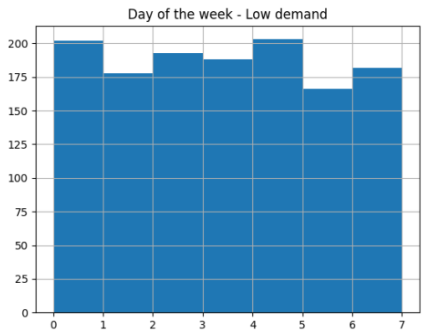
2 Data analysis

The given data consisted of 1600 data points (rows) and 15 features (columns). The column labelled "increase_stock" showed whether the demand for bikes was high or low given any row. The number of instances where the demand for bikes was high was only 288, which contrasted greatly to the remaining 1312 data points at which the demand was low. This imbalance was tackled using the method SMOTE on training data, which creates additional synthetic data points in the minority class (i.e. high demand) using the k-nearest neighbour algorithm [1] (see Section 3). Furthermore, it was decided that the F2 score was the most relevant factor for predicting bike demand since it weighs recall higher, meaning we value true positives higher than true negatives.

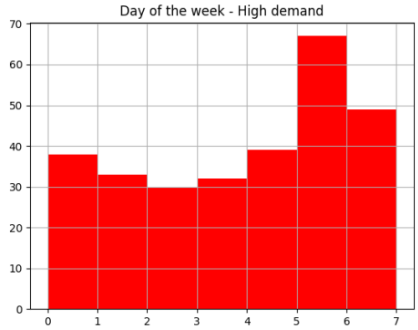
2.1 Features and what to include

Of the given features, the categorical features were the temporal related (along with "increase_stock") while the numerical were given by the meteorological ones. A handful of these features could be removed right away, namely "weekday", "summertime", "snow", "dew", "visibility" and "holiday". The column "snow" only consisted of zeros and provided no valuable data, while the other removed columns were excluded since they had some sort of dependency on other features. E.g. "dew" is directly correlated to "temp" and "humidity" and is not necessary to include. Furthermore, the columns for hours, days and months were transposed into two new columns respectively with coordinates in the unit circle. This made these features cyclic, which was a necessity for the k-nearest neighbour method (see Section 3.2). In order to get some further intuition about how the different features relate to each other and the bike demand, a variety of plots were created to draw some conclusions.

2.2 Hours, days, weeks and months



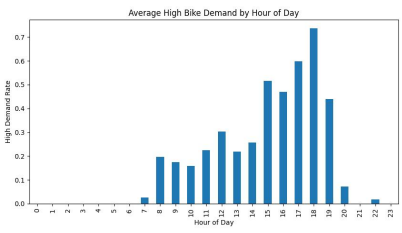
(a) The distribution of the weekdays when the demand is low.



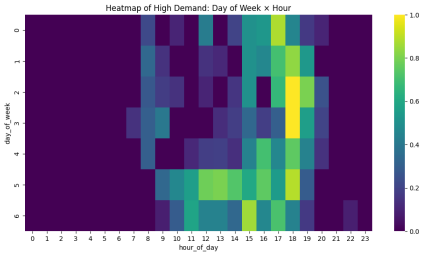
(b) The distribution of the weekdays when the demand is high.

Figure 1: Two histograms of the distribution of the days of the week for high and low demand of bicycles.

For the feature "day of the week", we have a much more even distribution. Generally, there is not a day that is drastically more likely to have low demand for bicycles. When we look at the days where we have high demand, we see that it is more likely to be during the weekend. This means that this feature might be significant when training our models.



(a) This plot shows the hour of day in the x-axis and plots it against the average high demand on that hour in the y-axis. Here we can clearly see what hours the demand for bicycles become higher.



(b) A heat map where we plot the day of week against the hour of day where each rectangle shows the rate of high demand on that day x hour

Figure 2: Two different plots that show the demand depending on the hour of day.

From both plots we can conclude that there is a clear range in the hour of day where the demand is much higher and there is low demand in the early morning and during the night.

2.3 Temperature and humidity

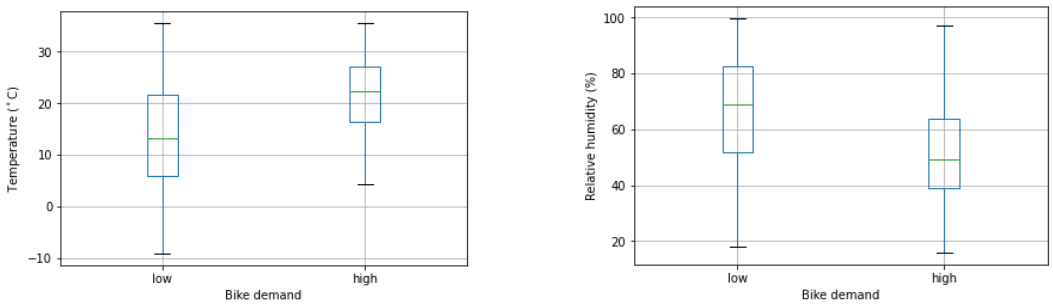


Figure 3: Temperature and humidity against bike demand

Temperature and humidity were plotted against the demand for bikes in two separate box plots. Both plots show clear trends where higher temperatures have higher demand and higher humidity have lower demand for bikes.

2.4 Wind speed and cloud cover

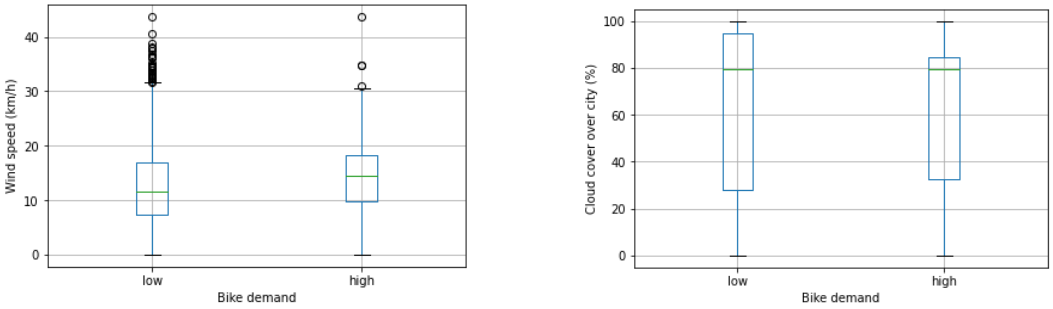


Figure 4: Wind speed and cloud cover against bike demand

The wind speed and cloud cover were plotted against the demand for bikes in respective box plots. Both plots have medians relatively close to each other compared to the previous weather-plots. The number of outliers when the wind speed is greater than 30 km/h is significantly higher when there is low demand. Recall that the number of data points at which the demand is low is significantly higher than the number of data points at which the demand is high.

2.5 Precipitation and snow depth

The number of instances at which there were precipitation and/or snow depth were plotted against the bike demand in two bar plots. Since the majority of data points have low demand, these results were rescaled w.r.t. the total number of points with low and high demand respectively.

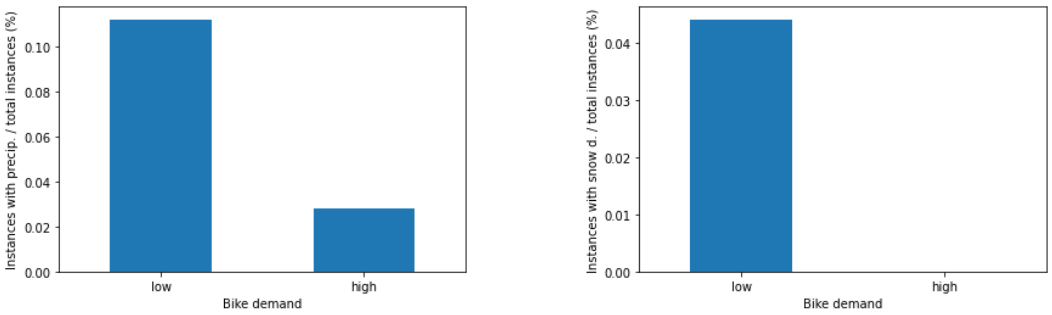


Figure 5: Precipitation and snow depth against bike demand

The plots show that the demand for bikes is generally low whenever there's precipitation in the city. Whenever there's snow depth the demand is always low.

3 Model development

The following 4 models were chosen for this report:

1. Logistic regression
2. K-nearest neighbour
3. Random forest
4. Boosting

In order to evaluate each model independently and then with each other, the data was split into training and testing data using Scikit-learn's "train_test_split" with seed 42. As addressed in Section 2, SMOTE was used on the training data to make it more balanced. Additionally, the training data was further split in grid search to determine the most suitable hyperparameters for each model independently. The scoring for the grid search was based on the F2-score. To have something to compare the models to, we chose a naive classifier that always predicts high demand, since we think an increase in false positives is less detrimental than an increase in false negatives.

3.1 Logistic regression

3.1.1 Mathematical summary

Logistic regression is a model used to predict categorical values. We will consider classifying a binary event, like for example "yes, no". For simplicity we map our events to $1, -1$. Logistic regression does this by fitting a logistic curve to the data. The logistic function maps the real line to $(0, 1)$ such that the output can be interpreted as a probability.

$$g(\mathbf{x}; \theta) = \frac{\exp(x \cdot \theta)}{1 - \exp(x \cdot \theta)}$$
$$\mathbf{x} = (1, x_1, x_2, ...x_k)$$
$$\theta = (\theta_0, \theta_1, ...\theta_k).$$

In other words, the model space is the set of logistic functions, and logistic regression finds the best fit within the model space with respect to the loss which in our case is logistic loss

$$L(\mathbf{x}, y; \theta) = \log(1 + \exp(-y(\mathbf{x} \cdot \theta))). \tag{1}$$

If we then minimize the average loss over the dataset we arrive at

$$\theta_{train} = \text{argmin}_\theta \frac{1}{n} \sum_{i=1}^{n} \log(1 + \exp(-y_i(\mathbf{x}_i \cdot \theta))) + \lambda ||\theta||_2.$$

The λ term is a regularization that "discourages" θ from growing too large. This expression does not have a closed form expression in general so we estimate it by numerical methods, e.g. gradient descent. One can pick a decision threshold t, such that if $g(\mathbf{x};\theta) \geq t$ then the model predicts 1, and -1 otherwise. Summarized, it is a model that tries to find the modified logistic function which describes the data most accurately.

3.1.2 Implementation and parameter selection

Using 10-fold cross validation on the training data we get the best $\frac{1}{\lambda} = \frac{1}{2^6} = \frac{1}{64}$ on the grid $[2^{-10}, 2^{-9}, ..., 2^9, 2^{10}]$. The decision threshold was chosen to be 0.5 since it misclassified the least on average [3].

3.2 K-nearest neighbour

The k-nearest neighbour algorithm is a non-parametric supervised classifier [5]. The classes are decided by a vote for their closest neighbours.

3.2.1 Mathematical summary

Let $T = x_n \in R_{n=1}^{d^N}$ be a training set with M number of classes including N training samples in a d-dimensional feature space. The class label for x_n is denoted c_n. Then given a query point x we do the following:

1. Find k-nearest neighbours from the set T for x and let $\hat{T} = (x_i^{NN}, c_n^{NN})$ be the set of k-nearest neighbours for x. Then we can calculate the distance between these points. The class label of the query point x is predicted by the majority voting of its neighbours, as shown in equation (2).

2. The class label of the point x can be predicted by the plurality voting using this equation:

$$c' = \text{argmax}_c \sum_{i=1}^{n} \left(x_i^{NN}, c_i^{NN}\right) \cdot \mathbb{1}\left(c = c_i^{NN}\right), \tag{2}$$

where c a class label in the dataset and c_i^{NN} is the class label for its ith nearest neighbour. $\mathbb{1}(c = c_i^{NN})$ is an indicator function where c_i^{NN} is the same as class c, otherwise zero.

There are several ways to calculate distance for the KNN algorithm, but for this implementation Manhattan Distance was used because that was seen as most effective when using cross validation to see what parameters to use for KNN. You can calculate Manhattan Distance as following:

$$d(x, y) = \left(\sum_{i=1}^{n} \mid x_i - y_i|\right).$$

3.2.2 Implementation and parameter selection

Because both time and days were cyclic classes, meaning that the day 0 and day 6 are exactly close to each other as day 1, we needed to change the features to work with the distance for the knn algorithm. This was made by remaking both the hour, day and month feature to have polar coordinates, creating a "hour sin" and "hour cos" class for example. With splitting each of these features into two we could measure the exact distance between these points.

We also needed to perform feature scaling on the train and test data. This is specifically important for the KNN algorithm because it measures distance and otherwise the features with a large numeric range would dominate the result. By doing feature scaling we could give every feature a mean of 0 and a deviation of 1.

We applied Grid search and cross-validation on the model and got new optimal parameters

to run with. The number of neighbours should be 4 (k=4), p=1, distance as measurement. Here p=1 means that Manhattan distance was used after cross-validation. 10-fold cross validation was used to evaluate F2-score and accuracy when evaluating all the models with different parameters.

3.3 Random forest

3.3.1 Mathematical summary

The random forest algorithm is best explained first by explaining how decision trees work.

We divide the data set χ into L disjoint regions $R_1, R_2, ..., R_L$ where we fit a simple model to each region R_l. The simple model $f_l(\mathbf{x})$ is that we classify each region based on the majority of the values $\{y_i\}$ in that region. The more complicated part is how we choose to divide each region. We use binary tree partitioning.

1. At level k: check $\{\mathbf{x} : x_j \leq s_k\}$ or $\{\mathbf{x} : x_j > s_k\}$.
2. Repeat until you have reached desired tree depth.

But we need to evaluate where to place every cutoff for each level.

1. At level k: For each input $x_1, x_2, ..., x_d$.
2. Find a cut-off point s_k that minimizes the loss function: $s_k = \arg_s \min Q(s)$.
3. Keep the pair (x_j, s_k) that fits the best.[6]

That is how a single tree is generated based on the dataset. For random forest, we instead generate decision trees based on random parts of the data set and only use certain features at one time. Each tree would then have more variation because it is not as dependent on the construction of the original dataset. We then take the majority vote of all the different trees to make the final decision, which reduces the variance greatly.[2]

3.3.2 Implementation and parameter selection

For the implementation of this method, we have to select some parameters. This was done using Grid search based on the desired outcome to evaluate the model. Many different parameters were tested with multiple values and the parameter values with the best outcome were to have 400 random trees in the forest and the trees were limited to a depth of 10. When it evaluated the models with different parameters, it evaluated F2-score and accuracy using 10-fold cross validation.

3.4 Boosting

Boosting is a learning method that sequentially constructs a strong classifier by combining multiple weak ones. The one implemented in this project was based on the AdaBoost algorithm [4], which utilizes decision tree classifiers with a maximum depth of 1. These are commonly referred to as "stumps" and are going to act as our weak classifiers.

3.4.1 Mathematical summary

Given data X and y, let n be the number of data points and $y \in \{-1, 1\}$ in order for the algorithm to work properly. The boosting classifier also uses a weight variable $w_i^{(b)}$ with initial values $w_i^{(0)} = \frac{1}{n}$, $i = 1, ..., n$. For the first iteration $b = 1$, the stump $f^{(1)}$ is trained on the input data including the weights $w_i^{(0)}$. A *weighted misclassification error* $E^{(1)}$ is then calculated as

$$E^{(1)} = \sum_{i=1}^{n} w_i^{(0)} \mathbb{1}(y_i \neq f^{(1)}(X_i)), \tag{3}$$

where each $f^{(1)}(X_i)$ is a prediction made by the stump based on the input X_i [6]. This is then used to calculate the variable $\alpha^{(1)}$, which is going to act as a weight variable based on the stump's "performance".

$$\alpha^{(1)} = \frac{1}{2} \ln \left(\frac{1 - E^{(1)}}{E^{(1)}} \right). \tag{4}$$

Note that we expect each stump to perform better than a random classifier would (where $\mathbb{E}$["error of random classifier"] $= 0.5$), meaning that $E^{(1)} < 0.5 \Rightarrow \alpha > 0$. Finally, the weights are rescaled based on the errors of $f^{(1)}$ for the next iteration.

$$w_i^{(1)} = cw_i^{(0)} \cdot e^{-\alpha^{(1)} \cdot y_i f^{(1)}(X_i)}, \tag{5}$$

where c is a normalization constant. Note that whenever the stump makes a correct prediction, the updated weight becomes smaller. The point is to let the data points that were incorrectly classified attain a higher weight variable so that the following stump can "focus" its training on what the previous stump got wrong. After a set number of iterations B, the algorithm stops and the final prediction $f_{\text{boost}}(X_i)$ for any data point X_i is calculated as

$$f_{\text{boost}}(X_i) = \text{sign}\left\{\sum_{b=1}^{B} \alpha^{(b)} f^{(b)}(X_i)\right\}. \tag{6}$$

3.4.2 Implementation and parameter selection

While AdaBoost served as the foundation of the used algorithm, the parameter for the maximum depth of the stumps (denoted m) was allowed to be adjusted for better performance. This together with B where the two hyperparameters that were used in a grid search to find the most optimal pair (B, m). The result showed that the most optimal choices where $B = 200$ and $m = 1$.

3.5 Model selection

After implementing all 4 models and finding suitable hyperparameters with grid search on the training data, the test data was used to compare the performances of the models to one another. The factors that were chosen to determine the most suitable model were F2-score and accuracy. The results are shown in Table 1. The two best performing methods were random forest and boosting, both of which

Table 1: The results of the different models when tested on the testing data.

Model	F2-score	Accuracy
Naive classifier	48.08 %	15.62 %
Logistic regression	45.96 %	79.06 %
K-nearest neighbour	57.69 %	84.38 %
Random forest	63.46 %	86.25 %
Boosting	70.14 %	84.38 %

had similar accuracies. But since Random forest had a lower F2-score, Boosting was chosen as the model to be put into production.

4 Conclusion

While all models had similar accuracies (apart from the naive classifier), their main differences lied in the F2-score. Boosting had the highest F2-score, which could be explained by the algorithm's high capability to focus training on data points that are harder to classify. Logistic regression scored the lowest F2-score, which could be explained by a lower model complexity.

By choosing to focus on the F2-score primarily, we have decided that it is more important to have bikes available even if the demand turns out to be low rather than not being able to meet a high bike demand. However, if we had aimed to find the best model to save costs, random forest would have probably been more optimal since it had the highest accuracy. Other scoring factors could have also been considered, such as F1-score, which balances recall and accuracy more evenly.

References

[1] Handling imbalanced data for classification. https://www.geeksforgeeks.org/machine-learning/handling-imbalanced-data-for-classification/. Accessed: 2025-12-18.

[2] Random forest algorithm in machine learning. https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/. Accessed: 2025-12-08.

[3] F. L. T. B. Andreas Lindholm, Niklas Wahlström. Machine learning a first course for engineers and scientists, 2022.

[4] D. Zachariah. Boosting, 2025.

[5] D. Zachariah. K-nearest neighbour, 2025.

[6] D. Zachariah. Tree-based methods, bagging, boosting, 2025.

A Appendix

A.1 Plots

Plots for Figure 1a and 1b, as well as extra plots for demand during holidays or non-holidays.

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("C:/Users/Ebba/Downloads/SML_project_template/
    training_data_ht2025.csv")
df = pd.DataFrame(data)
df.replace('high_bike_demand', 1, inplace=True)
df.replace('low_bike_demand', 0, inplace=True)

# Plots of the distribution of the days when there is high and low
    demand.
bike_demand_day = df[['day_of_week', 'increase_stock']]
high_bike_day = bike_demand_day[bike_demand_day['increase_stock'] ==
    1]
low_bike_day = bike_demand_day[bike_demand_day['increase_stock'] == 0]
hist1 = high_bike_day.hist(column='day_of_week', bins=[0, 1, 2, 3, 4,
    5, 6, 7], color='red'); plt.title('Day of the week - High demand')
    ; plt.subplot(); plt.show()
hist2 = low_bike_day.hist(column='day_of_week', bins=[0, 1, 2, 3, 4,
    5, 6, 7]); plt.title('Day of the week - Low demand'); plt.subplot
    (); plt.show()

# Plots for demand depending on if it is a holiday or not.
bike_holiday = df[['holiday', 'increase_stock']]
true_holiday = bike_holiday[bike_holiday['holiday'] == 1]
false_holiday = bike_holiday[bike_holiday['holiday'] == 0]
hist3 = true_holiday.hist(column='increase_stock', bins=3, color='red'
    ); plt.title('Demand - Holiday'); plt.subplot(); plt.show()
hist4 = false_holiday.hist(column='increase_stock', bins=3); plt.title
    ('Demand - No holiday'); plt.subplot(); plt.show()
```

Hour of day plot and day of week x hour of day heat map in Figure 2.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn.neighbors as skl_nb
#import sklearn
df = pd.read_csv("training_data_ht2025.csv")
df['high_demand'] = (df['increase_stock'] == 'high_bike_demand').
    astype(int)

hourly = df.groupby("hour_of_day")["high_demand"].mean()
hourly.plot(kind="bar", figsize=(10,5))
plt.title("Average High Bike Demand by Hour of Day")
plt.xlabel("Hour of Day")
plt.ylabel("High Demand Rate")
plt.show()


#HEATMAP
pivot = df.pivot_table(
    values="high_demand",
    index="day_of_week",
    columns="hour_of_day",
    aggfunc="mean"
)

```

```
plt.figure(figsize=(12,6))
sns.heatmap(pivot, cmap="viridis")
plt.title("Heatmap of High Demand: Day of Week    Hour")
plt.show()
```

Code for the figures in sections 2.3, 2.4 and 2.5.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv(r"training_data_ht2025.csv")

# Assigning numerical values to the high/low demand
df["increase_stock"] = df["increase_stock"].map({"low_bike_demand":0,
    "high_bike_demand":1})

# %% Temperature

ax = df.boxplot(by="increase_stock", column="temp")

ax.set_title("")
plt.suptitle("")
ax.set_xlabel("Bike demand")
ax.set_ylabel("Temperature ($^\circ$C)")
ax.set_xticklabels(["low", "high"])

plt.show()

# %% Precipitation

df_precip = df[df["precip"] > 0]

precip_counts = df_precip["increase_stock"].value_counts()

total_counts = df["increase_stock"].value_counts()

scale = precip_counts/total_counts


print(precip_counts)
print(total_counts)
print(scale)

ax = scale.plot(kind="bar")

ax.set_title("")
ax.set_xlabel("Bike demand")
ax.set_ylabel("Instances with precip. / total instances (%)")

ax.set_xticklabels(["low", "high"], rotation=0)

plt.show()

# %% Wind speed

ax = df.boxplot(by="increase_stock", column="windspeed")

ax.set_title("")
plt.suptitle("")
ax.set_xlabel("Bike demand")
ax.set_ylabel("Wind speed (km/h)")
ax.set_xticklabels(["low", "high"])

plt.show()

```

```
# %% Humidity

ax = df.boxplot(by="increase_stock", column="humidity")

ax.set_title("")
plt.suptitle("")
ax.set_xlabel("Bike demand")
ax.set_ylabel("Relative humidity (%)")
ax.set_xticklabels(["low", "high"])

plt.show()

# %% Snow depth

df_precip = df[df["snowdepth"] > 0]

ct = (df_precip["increase_stock"].value_counts()/1312).reindex([0, 1],
     fill_value=0)

ax = ct.plot(kind="bar")

ax.set_xlabel("Bike demand")
ax.set_ylabel("Instances with snow d. / total instances (%)")

ax.set_xticklabels(["low", "high"], rotation=0)

plt.show()

# %% Cloud cover

ax = df.boxplot(by="increase_stock", column="cloudcover")

ax.set_title("")
plt.suptitle("")
ax.set_xlabel("Bike demand")
ax.set_ylabel("Cloud cover over city (%)")
ax.set_xticklabels(["low", "high"])

plt.show()
```

A.2 Logistic regression

```
import pandas as pd
import numpy as np
import sklearn
from imblearn.over_sampling import SMOTE



high_low = np.vectorize(lambda x: 1 if x == 'high_bike_demand' else
    -1)

df = pd.read_csv("training_data_ht2025.csv")
df['increase_stock'] = high_low(df['increase_stock'])
features = df.drop(columns=['increase_stock','snow','weekday','
    summertime','dew', 'visibility','holiday'])


features["hour_sin"] = np.sin(2*np.pi*features["hour_of_day"]/24)
features["hour_cos"] = np.cos(2*np.pi*features["hour_of_day"]/24)

features["day_sin"] = np.sin(2*np.pi*features["day_of_week"]/7)
features["day_cos"] = np.cos(2*np.pi*features["day_of_week"]/7)

```

```
features["month_sin"] = np.sin(2*np.pi*features["month"]/12)
features["month_cos"] = np.cos(2*np.pi*features["month"]/12)

features = features.drop(columns=["hour_of_day"])
features = features.drop(columns=["day_of_week"])
features = features.drop(columns=["month"])


X_train, X_test, Y_train, Y_test = sklearn.model_selection.
    train_test_split(features, df['increase_stock'], test_size=0.2,
    random_state=42)

smote = SMOTE(sampling_strategy='auto', random_state=42)
X_train_resampled, Y_train_resampled = smote.fit_resample(X_train,
    Y_train)

parameters = {'C':[np.pow(2.0,a) for a in range(-10, 11)],
        'solver':['liblinear'],
        'penalty':['l2']}

fbeta = sklearn.metrics.make_scorer(sklearn.metrics.fbeta_score, beta
    =2, pos_label=1)
grid = sklearn.model_selection.GridSearchCV(estimator=sklearn.
    linear_model.LogisticRegression(max_iter=4000),
                                            param_grid=parameters,
                                            scoring=fbeta,
                                            cv=10,
                                            n_jobs=1)

grid.fit(X_train_resampled, Y_train_resampled)
cvParams = grid.best_params_
best_model = grid.best_estimator_


Y_predict = best_model.predict(X_test)

test_accuracy = sklearn.metrics.accuracy_score(Y_predict, Y_test)
test_f2 = sklearn.metrics.fbeta_score(Y_predict, Y_test,beta=2,
    pos_label=1)

print(f'parameters:{grid.best_params_}')
print(f'best (f2)score on training data:{grid.best_score_}')
print(f'accuracy:{test_accuracy}')
print(f'f2:{test_f2}')
```

A.3 K-nearest neighbour

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn.neighbors as skl_nb
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report,
    make_scorer, recall_score, fbeta_score
from sklearn.model_selection import GridSearchCV

df = pd.read_csv("training_data_ht2025.csv")

df['high_demand'] = (df['increase_stock'] == 'high_bike_demand').
    astype(int)

# Remake hour and week feature (cyclic)
df["hour_sin"] = np.sin(2*np.pi*df["hour_of_day"]/24)
df["hour_cos"] = np.cos(2*np.pi*df["hour_of_day"]/24)
```

```
17
18 df["dow_sin"] = np.sin(2*np.pi*df["day_of_week"]/7)
19 df["dow_cos"] = np.cos(2*np.pi*df["day_of_week"]/7)
20
21 df["month_sin"] = np.sin(2*np.pi*df["month"]/12)
22 df["month_cos"] = np.cos(2*np.pi*df["month"]/12)
23
24 # non-linear in weather
25 df["temp_humidity_index"] = df["temp"]*(1-df["humidity"]/100)
26
27 # Drop old non-cyclic hour,day,month
28 df = df.drop(["hour_of_day","day_of_week","month"], axis=1)
29
30 # other features to remove
31 df = df.drop(["snow","snowdepth","dew","windspeed","cloudcover","
       holiday"],axis=1)
32
33 from imblearn.over_sampling import SMOTE
34 from sklearn.model_selection import train_test_split
35
36 df = df.drop(['increase_stock'],axis=1)
37
38 features = df.drop(columns = ['high_demand'])
39 X_train, X_test, y_train, y_test = train_test_split(features, df['
       high_demand'], test_size=0.2, random_state=42)
40
41 smote = SMOTE(sampling_strategy="auto",random_state=42)
42 X_train_features, y_train_resampled = smote.fit_resample(X_train,
       y_train)
43
44 scaler = StandardScaler()
45 X_train_scaled = scaler.fit_transform(X_train_features)
46 X_test_scaled = scaler.transform(X_test)
47
48 fbeta_scorer = make_scorer(fbeta_score,beta=2, pos_label=1)
49
50 params = {
51     "n_neighbors": range(1,50),
52     "weights":["uniform","distance"],
53     "p":[1,2]
54 }
55
56 grid = GridSearchCV(
57     skl_nb.KNeighborsClassifier(),
58     params,
59     cv=10,
60     scoring=fbeta_scorer
61 )
62
63 grid.fit(X_train_scaled, y_train_resampled)
64
65 best_model = grid.best_estimator_
66
67 print("Best parameters:", grid.best_params_)
68 print("Best score:", grid.best_score_)
69
70 y_pred = best_model.predict(X_test_scaled)
71
72 test_f2 = fbeta_score(y_test,y_pred,beta=2,pos_label=1)
73 test_accuracy = accuracy_score(y_test,y_pred)
74
75 print("ACCURACY",test_accuracy)
76 print("F2SCORE",test_f2)
```

A.4 Random forest (and naive classifier)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score, KFold, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
import seaborn as sns
from imblearn.over_sampling import SMOTE
from sklearn.metrics import make_scorer, fbeta_score, accuracy_score, confusion_matrix

data = pd.read_csv("C:/Users/Ebba/Downloads/SML_project_template/training_data_ht2025.csv")
df = pd.DataFrame(data)
df.replace('high_bike_demand', 1, inplace=True)
df.replace('low_bike_demand', 0, inplace=True)

# Making hour, day and month cyclic
df["hour_sin"] = np.sin(2*np.pi*df["hour_of_day"]/24)
df["hour_cos"] = np.cos(2*np.pi*df["hour_of_day"]/24)

df["day_sin"] = np.sin(2*np.pi*df["day_of_week"]/24)
df["day_cos"] = np.cos(2*np.pi*df["day_of_week"]/24)

df["month_sin"] = np.sin(2*np.pi*df["month"]/24)
df["month_cos"] = np.cos(2*np.pi*df["month"]/24)

df = df.drop(columns=["hour_of_day"])
df = df.drop(columns=["day_of_week"])
df = df.drop(columns=["month"])

# Dropping features described in report.
df = df.drop(columns=['snow', 'weekday', 'summertime', 'dew', 'visibility', 'holiday'])

# Dividing the dataset.
X = df.drop(columns = ['increase_stock'])
y = df['increase_stock']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Implements SMOTE to account for the imbalance in the dataset.
X_train_res, y_train_res = SMOTE(sampling_strategy='auto', random_state=42).fit_resample(X_train, y_train)

f2_scorer = make_scorer(fbeta_score, beta=2)

param_grid = {'n_estimators': [100, 200, 400],'max_depth': [None, 10, 20],'min_samples_split': [2, 5, 10],'min_samples_leaf': [1, 2],'bootstrap': [True, False]}

grid = GridSearchCV(RandomForestClassifier(random_state=42),param_grid,scoring=f2_scorer, n_jobs=-1)
grid.fit(X_train_res, y_train_res)
print("Best params:", grid.best_params_)
print("Best F2 score:", grid.best_score_)

classifier = RandomForestClassifier(n_estimators=400, min_samples_leaf=1, min_samples_split=2, bootstrap=False, max_depth=10, random_state=42)
classifier.fit(X_train_res, y_train_res)

y_pred = classifier.predict(X_test)
```

```
y_naive = np.ones(len(y_test)) #Naive classifier.

naive_accuracy = accuracy_score(y_test, y_naive)
naive_f2_score = fbeta_score(y_test, y_naive, beta=2)
model_accuracy = accuracy_score(y_test, y_pred)
model_f2_score = fbeta_score(y_test, y_pred, beta=2)


# F2-score and accuracy for random forest classifier.
print(f'F2 score - random forest classifier: {model_f2_score * 100:.2f}%')
print(f'Accuracy - random forest classifier: {model_accuracy * 100:.2f}%')
# F2-score and accuracy for the naive classifier.
print(f'F2 score - naive classifier: {naive_f2_score * 100:.2f}%')
print(f'Accuracy - naive classifier: {naive_accuracy * 100:.2f}%')

# Using cross-validation to evaluate performance.
splits = 10
kf = KFold(n_splits=splits, shuffle=True, random_state=42)
cross_val_results_f2 = cross_val_score(classifier, X_train, y_train, cv=kf, scoring=f2_scorer)
cross_val_results_acc = cross_val_score(classifier, X_train, y_train, cv=kf, scoring='accuracy')

# NOTE: This is not the values that are compared to the rest of the models.
print("Cross-Validation Results (F2-score):")
for i in range(splits):
    print(f"  Fold {i+1}: F2 - {cross_val_results_f2[i] * 100:.2f}%, Accuracy - {cross_val_results_acc[i] * 100:.2f}%")
print(f'Mean F2-score: {cross_val_results_f2.mean()* 100:.2f}%')
print(f'Mean Accuracy: {cross_val_results_acc.mean()* 100:.2f}%')

# Heat map for the random forest classifier.
conf_matrix_pred = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_pred, annot=True, fmt='g', cmap='Blues', cbar=False)
plt.title('Confusion Matrix Heatmap - random forest classifier')
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.show()

# Heat map for the naive classifier.
conf_matrix_naive = confusion_matrix(y_test, y_naive)
sns.heatmap(conf_matrix_naive, annot=True, fmt='g', cmap='Reds', cbar=False)
plt.title('Confusion Matrix Heatmap - naive classifier')
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.show()
```

A.5 Boosting and model prediction

The class for the boosting algorithm was inspired by the one implemented in the boosting notebook from the course [4].

```
# Boosting

import pandas as pd
import numpy as np

df = pd.read_csv(r"training_data_ht2025.csv")

```

```
# Assigning -1 and 1 to bike demand
df["increase_stock"] = df["increase_stock"].map({"low_bike_demand":-1,
    "high_bike_demand":1})

# Removing "snow" column since every value is zero
df = df.drop(columns=["snow"])

# Removing additional columns due to dependency
df = df.drop(columns=["weekday"])
df = df.drop(columns=["summertime"])
df = df.drop(columns=["dew"])
df = df.drop(columns=["visibility"])
df = df.drop(columns=["holiday"])

# Making hour, day and month cyclic

df["hour_sin"] = np.sin(2*np.pi*df["hour_of_day"]/24)
df["hour_cos"] = np.cos(2*np.pi*df["hour_of_day"]/24)

df["day_sin"] = np.sin(2*np.pi*df["day_of_week"]/7)
df["day_cos"] = np.cos(2*np.pi*df["day_of_week"]/7)

df["month_sin"] = np.sin(2*np.pi*df["month"]/12)
df["month_cos"] = np.cos(2*np.pi*df["month"]/12)

df = df.drop(columns=["hour_of_day"])
df = df.drop(columns=["day_of_week"])
df = df.drop(columns=["month"])

# Boosting class

from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn import tree

class Boosting(BaseEstimator, ClassifierMixin):
    def __init__(self, B=10, m=1): #Default values - these are
    overwritten as the gridsearch looks for better parameters
        self.B = B
        self.m = m

    def fit(self, X, y):

        X = np.asarray(X)
        y = np.asarray(y).flatten()

        n = len(y)

        self.stumps = []
        self.alpha_list = np.array([])
        w = np.repeat(1/n, n)
        for b in range(1, self.B+1):
            f_model = tree.DecisionTreeClassifier(max_depth=self.m)
            f_model.fit(X,y, sample_weight=w)
            self.stumps.append(f_model)

            predictions = f_model.predict(X)

            mcs = abs(predictions - y)/2   # number of
    missclassifications
            E = sum(w*mcs)

            alpha = (1/2)*np.log((1-E)/E)
            self.alpha_list = np.append(self.alpha_list, alpha)

            w = w*np.exp(-alpha*predictions*y)
```

```
            w = w/sum(w) # normalizing

        return self

    def predict(self, X):

        X = np.asarray(X)

        preds = []
        for row in X:
            stump_preds = np.array([
                stump.predict(row.reshape(1, -1))[0]
                for stump in self.stumps
            ])

            boost = np.sum(self.alpha_list * stump_preds)
            preds.append(np.sign(boost))

        return preds

from sklearn.model_selection import train_test_split

# Seed: 42
features = df.drop(columns = ['increase_stock'])
X_train, X_test, y_train, y_test = train_test_split(features, df['
    increase_stock'], test_size=0.2, random_state=42)

# SMOTE
from imblearn.over_sampling import SMOTE

smote = SMOTE(sampling_strategy='auto', random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train,
    y_train)

# Grid-search
from sklearn.model_selection import GridSearchCV

model = Boosting()

param_grid = {
    'B': [50, 100, 200, 300],
    'm': [1, 2, 3]
}

# Using F2-scorer
from sklearn.metrics import make_scorer, fbeta_score
fbeta = make_scorer(fbeta_score, beta=2, pos_label=1)

grid = GridSearchCV(
    model,
    param_grid,
    scoring=fbeta,
    cv=10,
    n_jobs=-1
)

grid.fit(X_train_resampled, y_train_resampled)

print("Best params boosting:", grid.best_params_)
print("f2 score boosting:", grid.best_score_)

# Final evaluation
from sklearn.metrics import accuracy_score

```

```
best_model = grid.best_estimator_
prediction = best_model.predict(X_test)

test_accuracy = accuracy_score(y_test, prediction)
test_f2 = fbeta_score(y_test, prediction, beta=2, pos_label=1)

print("Final test accuracy:", test_accuracy)
print("Final test f2:", test_f2)

# PREDICTING RESULTS WITH BOOSTING METHOD ON TEST DATA

df = pd.read_csv(r"test_data_fall2025.csv")

# We handle the given data in the same way as we did the training data

df = df.drop(columns=["snow"])
df = df.drop(columns=["weekday"])
df = df.drop(columns=["summertime"])
df = df.drop(columns=["dew"])
df = df.drop(columns=["visibility"])
df = df.drop(columns=["holiday"])

df["hour_sin"] = np.sin(2*np.pi*df["hour_of_day"]/24)
df["hour_cos"] = np.cos(2*np.pi*df["hour_of_day"]/24)

df["day_sin"] = np.sin(2*np.pi*df["day_of_week"]/7)
df["day_cos"] = np.cos(2*np.pi*df["day_of_week"]/7)

df["month_sin"] = np.sin(2*np.pi*df["month"]/12)
df["month_cos"] = np.cos(2*np.pi*df["month"]/12)

df = df.drop(columns=["hour_of_day"])
df = df.drop(columns=["day_of_week"])
df = df.drop(columns=["month"])

# Predict on testing data
prediction_test = best_model.predict(df)

# Convert every -1 to 0
prediction_test = [0 if x == -1 else 1 for x in prediction_test]

# Save the values in a csv file
pd.DataFrame([prediction_test]).to_csv("predictions.csv", index=False,
    header=False)
```